

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – Mock Interview

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 3 – Mock Interview
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	Sk. Nuha	Reg. No.:	192472242
Section / Batch:	CSE-AI	Date:	

Instructions to Students

- Attend the mock interview individually and answer all technical questions clearly.
- Explain the logic, algorithm design steps, and complexity analysis wherever required.
- Demonstrate understanding of backtracking, recursion, and combinatorial problem-solving approaches.
- Differentiate clearly between P, NP, NP-Hard, and NP-Complete problems with suitable examples.
- Maintain clear communication, confidence, and logical presentation throughout the interview.

Question Type	Focus Area	Questions	Marks
Mock Interview	Demonstrate problem-solving skills in backtracking techniques and complexity classes through technical discussion and algorithmic analysis.	Q1- Q15	100
GRAND TOTAL:			100 Marks

SECTION A – Mock Interview

1. Explain how constraint ordering affects the efficiency of backtracking algorithms.
2. Compare the complexity of generating combinations vs permutations.

Code of Conduct

I Sk. Nuha-192472242 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sk. Nuha

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Knowledge	25%	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25%	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15%	Analyzes questions critically and provides well structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15%	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	10%	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10%	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Marks Evaluation:

Q.No	Technical Knowledge (25%)	Problem Solving (25%)	Analytical Thinking (15%)	Communication (15%)	Confidence (10%)	Response to Questions (10%)	Total Marks (100)
Q1							
Overall Total (100)							
Faculty In-charge			Course Coordinator			Head of Department	
Signature: _____			Signature: _____			Signature: _____	
Date: _____			Date: _____			Date: _____	

Q1. Explain How Constraint Ordering Affects the Efficiency of Backtracking Algorithms

Aim

To study the impact of constraint ordering on the efficiency of Backtracking algorithms.

Problem Statement

Backtracking algorithms explore possible solutions by making choices and undoing them when constraints are violated. The order in which constraints are checked can significantly affect the number of search states explored and the overall execution time.

Explanation

Constraint ordering determines the sequence in which conditions are evaluated during the search process.

Good Constraint Ordering

- Check the most restrictive constraints first.
- Invalid solutions are detected early.
- Large portions of the search space are pruned.

Poor Constraint Ordering

- Less important constraints are checked first.
- Invalid solutions are detected late.
- More unnecessary branches are explored.

Example

In the N-Queens problem:

1. Check column conflicts first.
2. Then check diagonal conflicts.

Early detection of conflicts reduces the number of recursive calls.

Effect on Efficiency

- Reduces search space.
- Increases pruning effectiveness.
- Decreases execution time.
- Improves scalability for large problems.

Complexity

- Worst Case: Remains exponential.
- Practical Performance: Significantly improved due to early pruning.

Conclusion

Constraint ordering does not change the theoretical complexity of backtracking algorithms but greatly improves practical efficiency by reducing unnecessary exploration of invalid solutions.

Q2. Compare the Complexity of Generating Combinations vs Permutations

Aim

To compare the computational complexity of generating combinations and permutations.

Problem Statement

Combinations and permutations are commonly used in combinatorial problems. The objective is to analyze their complexity and understand the difference in the number of generated solutions.

Combinations

Definition

A combination selects items without considering order.

Example:

{A,B}, {A,C}, {B,C}

Number of Combinations

Complexity

$O(nCr)$

Permutations

Definition

A permutation selects items where order matters.

Example:

AB, BA, AC, CA, BC, CB

Number of Permutations

Complexity

$O(nPr)$

Comparison Table

Feature	Combinations	Permutations
---------	--------------	--------------

Feature	Combinations	Permutations
Order Matters	No	Yes
Formula	nCr	nPr
Number of Results	Smaller	Larger
Time Complexity	$O(nCr)$	$O(nPr)$
Memory Usage	Lower	Higher

Example

For $n = 4$ and $r = 2$:

Combinations:

AB, AC, AD, BC, BD, CD

Total = 6

Permutations:

AB, BA, AC, CA, AD, DA,
BC, CB, BD, DB, CD, DC

Total = 12

Conclusion

Generating permutations is more computationally expensive than generating combinations because permutations consider different orders of the same elements. Therefore, permutation algorithms generally require more time and memory than combination algorithms.